Tomato Ripeness Detection and Counting on an Edge Device: How Annotation, Occlusion, Confidence Threshold and Hardware Affect the Reported Error

David Kalandadze
Supervisor: Brian McGinley

***Abstract*—Tomato maturity detectors are usually compared using one value: mean average precision (mAP) on one dataset. A counting system deployed on an edge device is also affected by factors that mAP does not record. These include the annotation team, the visible fraction of each fruit, the confidence threshold used to produce a count, the lighting, the model size, and the inference hardware. This work measures the effect of each factor on YOLO11 detectors trained on two public greenhouse datasets, Laboro Tomato and tomatOD. We map both datasets onto the same three maturity stages and deploy the detectors with Core ML on the Neural Engine of an iPhone 17 Pro. The datasets place the boundary between half-ripe and fully ripe fruit at different colours: 98% of tomatOD's fully ripe boxes fall within the colour range that Laboro labels half-ripe. A detector trained on both datasets retains its accuracy on each dataset and improves transfer in both directions. As a larger fraction of each fruit is hidden, detection recall falls from 0.884 to 0.057 at 70% hidden. Among the fruit still detected, the assigned stage is correct for 86% to 94% of them. The matching rule and the appearance of the occluder each change recall by about ten points. Selecting the confidence threshold reduces cross-dataset counting error from 3.29 to 1.34 fruit per image and stage without changing mAP. Across the five YOLO11 sizes trained under one protocol, mAP stops improving at the medium model, although package size and latency continue to increase. On the smartphone, YOLO11x is the largest size that still runs at 30 frames per second, with almost no margin. On a development laptop, the smallest model runs 3.85 times faster than on the smartphone, while the largest runs only 1.21 times faster. Laptop timings therefore overstate the speed advantage of small models. Three stops of over-exposure reduce the best model's recall by 0.205. This loss is more than three times the 0.058 recall difference between the smallest and largest models, and wider training-time colour augmentation does not recover it. Reproducing a reported counting error therefore requires the annotation source, occlusion and matching rules, confidence threshold, lighting conditions, and deployment configuration to be reported.**

***Index Terms*—agricultural computer vision, object detection, tomato ripeness, edge inference, on-device deep learning, occlusion, domain shift, calibration, illumination robustness, model scaling, Core ML**

I. Introduction

A grower deciding when to send pickers into a greenhouse needs to know, for each row, how many fruit are ready now and how many will be ready next week. Harvest scheduling and short-term yield forecasting both rely on these per-stage counts. In the longer term, an automated harvesting robot can use the same counts to decide which fruit to pick [1], [2]. Cameras can provide these counts because tomato maturity is judged almost entirely from colour, which can be measured from an image.

The deployment device affects both latency and reliability. Greenhouse rows often have poor network coverage. Sending every frame to a cloud server also adds a network round trip to each prediction, making latency higher and less predictable than on-device inference [3], [4]. A smartphone provides a camera and a dedicated neural accelerator. This work therefore assumes that inference runs on such an edge device and uses the camera frame rate as the time budget for the detector. Running convolutional detectors at camera frame rate on a smartphone has only recently become practical. Apple introduced its on-device inference framework, Core ML, in June 2017, and its first smartphone Neural Engine, in the A11 chip, in September of the same year [5], [6].

Public datasets with annotated greenhouse tomatoes and maturity labels have existed since 2020 [7], [8]. Detectors of the YOLO family [9], [10] train on them in under an hour, and studies published between 2020 and 2025 report mean average precision (mAP) values that improve on one another by a few points [11], [12], [13], [14], [15]. In almost all of these studies, a detector is trained on one dataset and its mAP is reported on that dataset's validation split.

A counting error reported in this way depends on at least six choices. The model is only one of them. First, annotation teams may use different meanings of *ripe*, so a model learns the colour boundary used in its training dataset. Second, leaves and neighbouring fruit conceal parts of many fruit. The construction of an occlusion test can therefore change its result. Third, a count includes every detection above a confidence threshold. The number of detections retained depends on the calibration of the model's confidence scores, as well as its ability to rank fruit [16]. Fourth, lighting changes the colour used to determine the stage. Fifth, larger models require more memory and inference time, which may exceed the resources of an edge device. Sixth, an exported model can use different numerical precision, preprocessing and hardware from the training checkpoint used for evaluation.

The author is with the Department of Computer Science, Atlantic Technological University, Galway, Ireland (e-mail: G00486658@atu.ie).

MSc in Computing dissertation, submitted October 2026. Supervisor: Brian McGinley.

Each choice can change the reported result while the trained weights remain fixed, yet papers usually report a single figure. Two papers can therefore report the same counting error for the same crop while using different measurement conditions. A speed measured on a development machine may also differ on the intended deployment device.

This work asks: *how much does the evaluation protocol contribute to a reported tomato counting error, how much does the model contribute, and what must be reported for the figure to be reproducible?* To answer these questions, we vary the annotation convention, the occluder, the confidence threshold, the lighting, the model size and the inference hardware one at a time. We keep the detector fixed wherever possible and measure the effect of each change.

Previous work has studied individual parts of this problem, but has not examined them together. Tomato maturity studies train and test on a single dataset and report mAP [12], [13], [14], [15]. Detectors for tomatoes have been run on Android phones [17], [18], on an NVIDIA Jetson board [19], and, for fruit detection without maturity stages, on the Neural Engine of an iPhone [20]; each of these reports the speed of one or a few models on one platform. To our knowledge, this is the first study that compares the two public tomato maturity datasets under one label set, and the first to report maturity detection accuracy, counting error and Neural Engine latency together for a complete YOLO11 model-size range (n to x) on a smartphone.

This work makes six contributions:

1) It measures how well detectors transfer between Laboro Tomato and tomatOD after both are mapped onto one three-stage label set, and explains the asymmetric errors by measuring where each dataset places its colour boundaries.
2) It separates detection, stage assignment and counting under controlled occlusion, and shows how the matching threshold and the occluder's appearance change the result.
3) It shows that a fixed confidence threshold can reverse a cross-dataset counting comparison in which mAP shows no difference.
4) It follows one trained model from FP32 PyTorch, through FP16 Core ML, to a physical iPhone, and checks that the laptop and the smartphone produce the same detections.
5) It trains all five YOLO11 sizes under one protocol and measures accuracy, counting error, package size and latency on a laptop and on the smartphone. Accuracy stops improving at the medium model (YOLO11m), while the largest model (YOLO11x) only just meets the real-time budget, so the two sizes above YOLO11m increase latency without improving accuracy.
6) It measures how much a change in exposure or white balance of the test images reduces accuracy, and whether a wider colour augmentation during training recovers the loss, with every configuration trained at three seeds so that real effects can be separated from training noise [21].

The rest of the paper is organised as follows. Section II reviews tomato maturity detection, the public datasets, how detectors are evaluated, and previous work on robustness, occlusion and on-device inference, and identifies the gap this work addresses. Section III describes the datasets, experimental protocols and statistical analysis. Section IV reports the six experiments. Sections V to VIII discuss the results, describe the software artefact and its limitations, and conclude.

II. Background and Related Work

This section first reviews public tomato maturity datasets and the detectors trained on them. It then explains detector evaluation and the distinction between a count and a detection score. The remaining subsections review work on the factors varied in this study: annotation conventions, robustness to lighting, occlusion, and inference on an edge device. Table I summarises the resulting research gap.

A. Tomato maturity datasets and detectors

Public annotated data for tomato maturity is recent. The two datasets used in this work were both released in 2020. *Laboro Tomato* was published by Laboro.AI, a Japanese company, and contains 804 images collected at a local farm with two different cameras, at 3024×4032 and 3120×4160 pixels [7]. It has bounding boxes and instance masks for six classes, which combine two fruit sizes with three maturity stages. The stages are defined by the share of the fruit surface that is red: at least 90% for fully ripened, 30–89% for half-ripened, and less than 30% for green, which follows the logic of the USDA colour classes [22]. *tomatOD* was published by Tsironis *et al.* [8] and contains 277 images taken over three days in April 2019 in a soilless greenhouse in Crete, Greece. The authors used a 12-megapixel machine-vision camera to simulate a camera mounted on a robot arm. Two agriculturists labelled its 2,418 boxes as unripe, semi-ripe or fully ripe. They deliberately left fruit unlabelled when more than 50% was occluded or when it was beyond the second plant row. Thus, the datasets were collected in different countries with different cameras, annotators, and definitions of ripeness.

Other public tomato datasets exist but are less suited to maturity counting. The Rob2Pheno dataset of Afonso *et al.* [23] contains 123 RGB-D images (720×1280) from a commercial greenhouse in the Netherlands. Its red and green labels were derived automatically from a colour ratio and were not annotated by hand. The TomatoPlantFactory dataset [24] contains 520 images of the Micro-Tom variety under artificial lighting in China, with two classes (green and red). A frequently used Kaggle dataset [25] has 895 images with a single "tomato" class and no maturity labels. Laboro Tomato and tomatOD are, to our knowledge, the only two public datasets with three hand-annotated maturity stages in commercial-style greenhouses, which is why this work uses them.

Researchers have applied several detector architectures to these datasets since 2020. YOLO-Tomato (2020) adapted YOLOv3 with dense blocks and circular boxes to detect tomatoes under changing light and occlusion, without classifying maturity [11]. Mu *et al.* (2020) detected highly occluded green tomatoes with Faster R-CNN and estimated their number [26],

and Afonso *et al.* (2020) detected and counted tomatoes with Mask R-CNN [23]. Later work focused on maturity: Su *et al.* (2022) classified four maturity classes with a MobileNet-based YOLOv3 [12]; Li *et al.* (2023) improved YOLOv5 for four maturity stages [13]; and a second Li *et al.* (2023) added attention to YOLOv8 for maturity detection and counting [14]. Camacho and Morocho-Cayamcela (2023) compared Mask R-CNN and YOLOv8 on a three-class version of Laboro Tomato [15]. We replicated their protocol as the starting point of this work. Each study trains and tests on a single dataset, usually its own, and reports mAP or precision and recall at one setting. We found no study that trains on Laboro Tomato and tests on tomatOD, or the reverse. The effect of the different annotation conventions has therefore not been measured before.

B. Maturity is a colour judgement, and its boundaries are conventions

A tomato ripens through a continuous change in pigment, from green chlorophyll through orange carotenoids to red lycopene. The change is gradual, so the discrete labels green, half-ripened and fully ripened divide a continuum. The fruit has no physical marker at the boundary between stages. Postharvest science measures this change with colour indices, most often the CIELAB a^* value or the a^*/b^* ratio [27], and standards for grading tomatoes by colour define stages by the share of the surface that has turned red [22].

A detector trained to name a stage learns the boundaries used by the people who annotated its training data. When it is evaluated against labels drawn by another team, its error therefore includes the disagreement between the two annotation conventions. Because the boundary is a colour threshold, illumination, camera white balance and over-exposed highlights also move a fruit across it. Section III-C measures where the two public datasets place these boundaries.

C. How detectors are evaluated

Most fruit detectors use a single-stage design: the network predicts a set of boxes, each with a class and a confidence score, in one forward pass. This design is generally less accurate than two-stage methods such as Mask R-CNN, but is fast enough for real-time use [9]. The YOLO family is therefore widely used in agricultural vision. This work uses YOLO11 [10], [28], whose five sizes (n, s, m, l, x) share one architecture and differ only in network width and depth. They therefore form a controlled series for measuring the effect of extra capacity. Before the boxes are returned, non-maximum suppression (NMS) removes duplicate boxes for the same object, keeping the most confident one [29].

Most studies compare detectors using mean average precision (mAP), as defined for the COCO benchmark [30]. A predicted box counts as correct when its intersection over union (IoU) with an annotated box exceeds a threshold. IoU is the ratio between the overlapping area of the two boxes and the total area covered by either box. Precision and recall are computed as the confidence threshold is lowered through the ranked list of predictions. The area under the precision–recall curve is averaged over classes and over IoU thresholds from 0.5 to 0.95 (mAP50–95), or taken at IoU 0.5 only (mAP50). Two properties of mAP matter for this paper. First, it averages over all confidence thresholds and describes how well the predictions are ranked. It does not describe the model's behaviour at the single threshold used in an application. Second, it is computed per box and therefore does not directly measure a count derived from the boxes.

D. Counting is a different task from detecting

A greenhouse application needs a fruit count. To produce one, the application counts every prediction above a chosen confidence threshold. The number of predictions above this threshold depends on the calibration of the model's confidence scores. Calibration describes how closely a score of 0.7 corresponds to a 70% chance of being correct, and modern networks are known to be poorly calibrated [16]. Detection errors can also cancel in the count. An image in which one fruit is missed and one false fruit is detected has a perfect count but two detection errors.

Detection quality and counting quality have been shown to diverge in practice. In a comparison of fruit detection and counting methods for yield mapping in apple orchards, the method with the best detection result differed from the method with the best count [31]. Reporting mAP without the count therefore leaves the error that matters to the application unmeasured.

E. Robustness to dataset and lighting changes

Computer-vision models often perform worse on a dataset other than the one used for training. Datasets can differ enough for a classifier to identify the source of an image, and models trained on one dataset can transfer poorly to others [32]. Robustness to common image changes, such as noise, blur, weather and changes in brightness, has since been benchmarked systematically for image classification, and models that rank well on clean images can rank differently under such changes [33]. White-balance errors in particular have been shown to reduce the accuracy of trained networks [34]. Data augmentation addresses this problem by adding synthetic variation that the training set lacks [35]. In the YOLO training pipeline, the colour part of this augmentation is a random change of hue, saturation and brightness within three fixed limits [36], [37]; Section IV-H tests whether widening those limits helps.

Agricultural images are especially exposed to lighting change. Greenhouse light varies with the time of day, the weather and the glazing, and the feature being classified, colour, is the one that lighting changes most directly. Previous agricultural work has addressed illumination mainly by changing the input representation or by collecting more varied data. Kirk *et al.* replace the RGB input with colour-opponent features and report better generalisation to a real strawberry farm than an RGB detector of comparable accuracy [38]. Haggag *et al.* vary dataset size, camera viewpoint and illumination for a tomato greenhouse detector and find illumination the hardest of the three shifts to generalise across [39]. Neither study tests the colour augmentation already present in the

training pipeline, and neither reports whether the differences it observes remain when the model is retrained with a different random seed. None of the tomato maturity studies reviewed in Section II-A evaluates its detector under controlled changes of exposure or white balance.

F. Evaluation under occlusion

Occlusion has been studied through part-based architectures, controlled benchmarks and synthetic augmentation. Context-aware CompositionalNets show that the context around an object becomes especially important when the object is strongly occluded [40]. Gilroy *et al.* argue that broad categories such as "partly occluded" hide useful information, and evaluate seven detectors at measured occlusion levels from 0% to 99% [41]. Kassaw *et al.* compare real and artificial occlusion and find that performance depends on the type of occluder [42]. During training, methods such as Cutout, Random Erasing and CutMix hide or replace rectangular image regions to make networks less dependent on any single part of an object [43], [44], [45].

In agriculture, Liao *et al.* place real leaves over apples and control the share of each fruit that is hidden [46]. This work follows the same principle, a known hidden fraction and an occluder taken from the same scene, but uses a rectangular strip of background, and adds two controls: a flat black strip, to measure the effect of the occluder's appearance, and the natural overlap between fruit, which needs no synthetic occluder at all.

G. Running the model on an edge device

A harvesting or scouting aid is used in the field, so its speed must be measured on its intended edge device. Running a model in the cloud requires a reliable network connection and adds a network round trip to every frame, which is often slower and less predictable than local inference [3], [4]. Reviews of edge intelligence in agriculture identify connectivity as the main constraint on practical systems and argue for on-device inference [47]. Deep learning on smartphones is recent: Core ML and the first iPhone Neural Engine appeared in 2017 [5], [6], and benchmarks of neural network inference on phones date from 2018–2019 [48].

Moving a model from a training workstation to a smartphone changes several parts of the inference pipeline at once. The weights are usually converted to 16-bit floating point; NMS and image preprocessing may be built into the exported model; execution moves to a vendor-specific neural accelerator with its own numerical behaviour; and long runs heat the device, which may cause the operating system to reduce its speed. In Core ML, the term compute units denotes the processors that the runtime is allowed to use, such as the CPU and Neural Engine. A latency figure can therefore only be interpreted together with the device, the permitted compute units, the operating-system version, and whether the run was short or sustained.

Edge deployment in agriculture has mostly been studied on single-board computers and dedicated accelerators. Mazzia *et al.* run a reduced YOLOv3 for apple detection on a Raspberry Pi with a neural compute stick and on two NVIDIA Jetson boards, and treat the frame rate on these boards as the main constraint [49]. Work on citrus compares architectures by their speed on the target hardware [50]. Model scaling has been studied as a design problem for YOLO detectors, varying width, depth and input resolution together against a hardware budget [51]. For tomatoes specifically, Zeng *et al.* (2023) quantised a lightweight YOLOv5 and ran it in an Android application at 26.5 frames per second [17]; TSE-YOLO (2025) runs ripeness detection and segmentation on an Android phone [18]; GAE-YOLO (2025) runs on an NVIDIA Jetson TX2 with a maturity assessment [19]; and Kafi *et al.* (2025) run a YOLOv8 tomato detector, without maturity stages, on the Neural Engine of an iPhone receiving video from a drone [20]. These studies report a throughput figure for one platform. This work keeps one exported model fixed and measures its latency across devices, compute-unit settings, operating-system versions and run lengths.

H. What a single benchmark number hides

Reported improvements vary across training runs. Retraining the same configuration with a different random seed changes the data order, the augmentation and the initial weights, and the resulting spread is often as large as the differences that papers claim; taking that variance into account changes which comparisons hold up [21]. Results from a single training run are still the norm in applied agricultural vision, which is why this work repeats the colour-augmentation experiments over three seeds and uses paired statistical tests (Section III-H).

I. Summary and research gap

Table I summarises which of the factors studied here previous tomato work has addressed. Each column is a factor that can change a reported counting error without any change to the model. Most previous studies report accuracy on one dataset with one model; counting, deployment on a device and seed variance each appear in only a few, and no study varies the annotation convention, the confidence threshold, or the lighting in a controlled way. This work evaluates a standard detector and measures how much each factor changes the result. The contribution is this controlled evaluation; no new detector is proposed.

III. Materials and Methods

A. Datasets and shared taxonomy

Two public datasets are used (Section II-A). Laboro Tomato has 643 training and 161 validation images, photographed at 3120×4160 or 3024×4032 pixels in portrait or landscape orientation. Its six classes combine two fruit sizes (large and small) with three maturity stages. tomatOD has 277 images, of which 55 form its validation split. Both datasets are used with their own published train/validation splits, so no image appears in both training and evaluation.

The class names of both datasets are mapped onto three stages: green, half-ripened and fully ripened. The conversion

TABLE I
FACTORS ADDRESSED BY PREVIOUS TOMATO DETECTION STUDIES AND BY THIS WORK. ✓: MEASURED OR REPORTED; (✓): PARTLY, E.G. DISCUSSED QUALITATIVELY OR FOR ONE MODEL ONLY; –: NOT ADDRESSED. "EDGE" MEANS THAT LATENCY WAS MEASURED ON THE TARGET DEVICE.

Study	Year	Maturity	Count	Cross-dataset	Occlusion	Lighting	Threshold	Edge
Liu *et al.* [11]	2020	–	–	–	(✓)	(✓)	–	–
Mu *et al.* [26]	2020	–	✓	–	✓	–	–	–
Afonso *et al.* [23]	2020	(✓)	(✓)	–	–	–	–	–
Su *et al.* [12]	2022	✓	–	–	–	–	–	–
Li *et al.* [13]	2023	✓	–	–	–	–	–	–
Li *et al.* [14]	2023	✓	✓	–	–	–	–	–
Camacho *et al.* [15]	2023	✓	–	–	–	–	–	–
Zeng *et al.* [17]	2023	–	–	–	–	–	–	✓
Haggag *et al.* [39]	2024	–	–	(✓)	–	✓	–	–
Jia *et al.* [18]	2025	✓	–	–	–	–	–	✓
Kafi *et al.* [20]	2025	–	–	–	–	–	–	✓
This work	2026	✓	✓	✓	✓	✓	✓	✓

uses class names because the numeric class identifiers are inconsistent between the datasets and between the splits of tomatOD (Section IV-A).

Two preprocessing steps are applied. About 40% of Laboro images carry an EXIF orientation tag. The stored pixels are in landscape orientation, but the annotations assume an upright image. The YOLO data loader ignores the tag, so the boxes do not align with the fruit. We rotate these images on disk before conversion to align the pixels and boxes. This correction reduced the ripe-fruit counting MAE of a six-class baseline detector from 0.93 to 0.82. We then convert the COCO annotations to the YOLO text format. No other preprocessing is done offline. Images are resized on the fly to an input size of 640 pixels: at evaluation and inference the longer side is scaled to 640 and the image is padded to a 640×640 square, which preserves the aspect ratio, while during training the standard Ultralytics augmentations (mosaic, scaling, translation, flipping and HSV colour jitter) are applied at the same input size.

Three YOLO11n detectors are trained for 60 epochs at 640 pixels, with batch size 16, seed 0, and no early stopping. One detector is trained on Laboro, one on tomatOD, and one on both datasets. We refer to the detector trained on both datasets as the *mixed* model. Each detector is evaluated separately on both validation splits.

B. Metrics

Detection quality is reported as COCO-style box mAP50–95 and mAP50 (Section II-C). Counting quality is reported as the mean absolute error between the predicted and the annotated number of fruit, per image and per maturity stage,

$$\mathrm{MAE}_{count} = \frac{1}{3N} \sum_{i=1}^{N} \sum_{s=1}^{3} |\hat{c}_{i,s} - c_{i,s}| \,. \tag{1}$$

Here, N is the number of images and $\hat{c}_{i,s}$ and $c_{i,s}$ are the predicted and annotated numbers of fruit of stage s in image i. Its unit is fruit per image and stage: a counting MAE of 1.25 means that, on average, the count for each stage in each image is off by 1.25 fruit. The predicted count is the number of detections of that stage above the confidence threshold.

For the occlusion experiments, detections are sorted from highest to lowest confidence and paired with annotated boxes without using the maturity labels. Each detection and annotation can be paired only once. This greedy matching separates detection recall from the stage accuracy of the matched detections. Unless stated otherwise, the confidence threshold is 0.25 and the matching threshold is IoU 0.5.

C. Colour analysis

The colour of a fruit is measured in the CIELAB colour space, which separates lightness (L^*) from two colour-opponent axes. The a^* axis runs from green (negative values) to red (positive values). It is a dimensionless coordinate defined by the CIE [52]; it has no formal bound, and for colours that an sRGB camera can record it lies roughly between -90 and $+100$. a^* and the related a^*/b^* ratio are standard colour indices for tomato ripening in postharvest science [27], because the change from chlorophyll green to lycopene red moves a fruit mainly along this axis.

For each annotated box, we calculate the median a^* in its central half. This region excludes most of the stem and leaves at the box edges. We convert the pixels from sRGB to CIELAB with the D65 white point. White balance and illumination affect the whole image, so we subtract the median a^* of the entire frame from every box in that frame. The frame median is dominated by foliage, and the resulting value is called background-corrected a^*. For each dataset, we use grid search to fit two cut points on this axis (green | half and half | fully). The search maximises balanced accuracy against the labels of that dataset. We then apply the cut points from each dataset to the boxes from the other dataset. The proportion of labels that change measures the disagreement between the annotation conventions.

D. Controlled and natural occlusion

We simulate occlusion by covering part of every annotated fruit with a rectangular strip. This intervention adapts the region-erasing methods used for augmentation and robustness testing [43], [44] to a test-time setting with a known hidden fraction, as used in controlled occlusion benchmarks [41]. The strip enters each ground-truth box from a randomly chosen



Fig. 1. Illustration of progressive synthetic occlusion on a representative red tomato: 0%, 20%, 40%, and 60% of its annotated box is covered from one side. The texture is sampled from a fruit-free region of the same frame. The straight boundary visualises the controlled intervention; quantitative results use the full evaluation set.

side and covers a set fraction of its area. Thus, "40% hidden" has the same definition for a large fruit and a small one. We fill the strip with a fruit-free background crop from the same image to preserve local lighting and colour statistics. A flat black strip provides an appearance control. The colour-directed variants cover either the redder or the greener half. We select that half from the median a^* along the axis with the greatest colour separation. The ground-truth boxes continue to describe the full fruit.

We estimate natural overlap separately from intersections between untouched ground-truth boxes. Because it omits leaves, this estimate is a lower bound on total natural occlusion. It is also likely to overestimate fruit-on-fruit coverage because rectangular boxes include background at their corners.

E. Model-size ladder

We train all five YOLO11 sizes under one protocol to measure the effect of model size directly. This avoids a comparison based on published scaling numbers from different protocols. YOLO11n, s, m, l, and x are trained on the Laboro split for 60 epochs at 640 pixels with seed 0 and nominal batch 64. Their physical batches, the numbers of images processed together in one training step, are 16, 16, 8, 8, and 4 respectively. These are the largest batches that fit in 24 GB of unified memory. Every checkpoint is exported to FP16 Core ML with non-maximum suppression (NMS) included in the exported model, using confidence 0.25 and IoU 0.45. A detector proposes many overlapping boxes for the same fruit; NMS keeps the most confident box and removes any other box that overlaps it by more than the IoU threshold [29]. Including it in the exported model means that the timed model is exactly the model that would be shipped in the application.

We then score each size in three ways. Detection quality is measured by mAP50–95 on the Laboro validation split. Counting quality is the best counting MAE over a sweep of the confidence threshold, reported for all stages together and for fully ripe fruit alone. Computational cost is measured by Core ML package size, model load time, and inference latency on both devices. We report the median and the p90, an upper-tail latency that captures slow frames as well as typical ones.

F. Photometric perturbation

We simulate test-time lighting changes by applying two single-parameter changes to the validation images. Exposure and white-balance shifts are common camera errors, and previous work has used both to test predictions under changed capture conditions [33], [34]; exposure errors in particular are usually simulated in steps of exposure value (stops) [53]. We apply both changes in linear light after undoing the sRGB gamma curve, then convert the result back to sRGB, following the standard practice of inverting the camera's processing before changing gains [54]. A gain applied this way corresponds to what the camera itself would have done, whereas the same gain applied to gamma-encoded pixel values would not. The exposure intervention multiplies every channel by 2^k for $k \in [-3, 3]$ stops. The white-balance intervention applies opposite gains to the red and blue channels, $1 \pm 0.35w$ for $w \in [-1.5, 1.5]$, while keeping green fixed. Positive w produces a warm cast and negative w a cool cast. Section III-C measures ripeness along a green-to-red axis. A colour cast therefore moves the apparent colour of every fruit along the axis used to assign its stage before the detector processes the image.

The intervention recolours the pixels without changing the ground-truth boxes. Exposure changes confidence scores across the predictions. We therefore run each level once with a minimum confidence of 0.05 and score the resulting predictions at every threshold in the sweep. The tables report a fixed threshold of 0.25 alongside the best threshold for that level. A comparison at only the fixed threshold would combine changes in calibration with changes in robustness, as shown across datasets in Section IV-C.

G. Training-time colour augmentation

During training, Ultralytics YOLO changes the colour of every training image by a random amount in HSV space, which represents hue, saturation, and value. This form of photometric augmentation was introduced for the YOLO family in YOLOv4 [36]. Three parameters set the maximum change. Their default values are $h = 0.015$, $s = 0.7$ and $v = 0.4$ [37]. This is the only colour augmentation in the pipeline. We compare four configurations while keeping everything else unchanged. The `stock` configuration uses the default recipe. The `hsv_v08` configuration doubles the brightness jitter to $v = 0.8$. The `hsv_wide` configuration uses $v = 0.8$ and widens hue to $h = 0.05$, which is the HSV change most similar to a colour cast. The `no_colour` configuration sets all three parameters to zero and provides the comparison with no colour augmentation.

The test-time perturbation uses a different photometric process from the HSV jitter used during training. An exposure change applied in linear light is generated differently from the training-time noise. The evaluation therefore tests whether the effect of HSV augmentation transfers to that distinct process.

H. Statistical analysis

We estimate training variation separately from evaluation variation. *Training variation* is estimated by repeating every colour-augmentation configuration at seeds 0, 1, and 2 under an identical protocol. We also repeat the `stock` configuration so that its variation across runs provides a noise floor for differences between recipes. Results are reported as the mean

over seeds with the observed range. We do not assume that three runs establish a normal distribution.

Evaluation variation is estimated at image level, where the sample is larger. For two models at one perturbation level, we calculate their difference in counting error for each of the 161 validation images. We resample these paired differences with 10,000 bootstrap replicates to obtain a 95% confidence interval. We also test the paired null hypothesis with a sign-flip permutation test that uses 10,000 relabellings of the same per-image differences. We use a permutation test because the per-image counting errors are small integers with many ties. The tie correction for a signed-rank statistic is approximate in this setting, whereas the sign-flip null is exact under the paired-exchangeability hypothesis. Detection recall is a ratio of pooled counts, not a per-image mean. Its interval therefore uses the same bootstrap applied to the per-image matched and ground-truth counts. When we test a family of levels together, we adjust the p-values across that family with the Holm correction.

We average each per-image series over seeds before the paired test. The resulting interval therefore refers to a recipe and not to one checkpoint.

We report both estimates. The bootstrap interval quantifies uncertainty on this validation set. The seed range shows how much of a difference remains after retraining.

I. Deployment protocol

The mixed detector is exported to FP16 Core ML with non-maximum suppression included, at confidence 0.25 and IoU 0.45 (Section III-E). Mac measurements use the same 161 images and scoring code for PyTorch and for each Core ML processor configuration. The smartphone benchmark uses the identical model package and three bundled 640×640 images over ten repetitions. It records median latency and p90 latency, an upper-tail value that captures slow frames as well as typical ones. It also records load time, model outputs, device identifier, OS, and thermal state.

The same benchmark software measures every size in the ladder on two devices: a Mac16,8 laptop and an iPhone 17 Pro (`iPhone18,1`). Each device uses two compute-unit settings. Compute units specify which processors Core ML may use; the settings are CPU only and CPU with the Neural Engine. We install a Release build of the companion application on the device and select the model under test through a launch argument. The report is then transferred from the device automatically, without manual transcription. The reported latency covers inference only. It excludes camera capture, display, and application rendering, so it is a lower bound on the cost of a live-camera pipeline.

We also measure sustained behaviour and cross-device agreement. The sustained test runs 4,500 consecutive frames on the Neural Engine. We report its median, p90, and the thermal state provided by the operating system at the end of the run. To check cross-device agreement, we score the same images on the laptop and the smartphone and match the two sets of boxes. This check establishes whether the devices produce the same detections before their timings are compared.

J. Edge-device implementation

The detectors are deployed in an iOS application written in Swift with SwiftUI, which runs entirely on the device and needs no network connection. Figure 2 shows the path from a trained checkpoint to a count on the screen.

Export. Each trained checkpoint is converted with the Ultralytics exporter to a Core ML package in 16-bit floating point, which halves the file size compared with 32-bit weights. NMS is included in the exported model, so the application receives final boxes with a class label and confidence score. The application therefore requires no separate post-processing code. The model metadata stores the class names, which the application reads when loading the model. The same code can therefore run the six-class Laboro models and the three-stage mixed model.

Inference. The application passes each image to the network through Apple's Vision framework. Vision scales the image to fit the 640×640 network input while keeping its aspect ratio, which matches the letterboxing used during training, and corrects for the camera orientation. The compute-unit setting specifies which processors Core ML may use: the CPU alone or the CPU together with the Neural Engine. This setting is fixed when the model is loaded, so the application creates one model instance for each setting. The GPU setting is excluded because the Core ML graph compiler failed on this model on both the device and the simulator. The GPU could therefore not be measured (Section VII).

Two modes of use. In the photo mode the user picks an image from the photo library or one of three bundled samples; the application draws the boxes on the image and shows the number of fruit per stage. In the live mode the rear camera delivers 1280×720 frames, which are processed one at a time on a background queue. When inference takes longer than the interval between frames, the application drops late frames instead of adding them to the queue. The boxes on the screen therefore belong to the most recent processed frame. The screen shows the number of fruit detected and the inference time for the latest frame. A menu selects any of the five YOLO11 sizes bundled with the application, and a switch turns the Neural Engine on or off.

Benchmark mode. The same application contains the benchmark used in Section IV-E. When launched with a command-line argument from the development machine, the application loads the selected model and runs the bundled images for a fixed number of repetitions under each compute-unit setting. It records the latencies, load time, detections, device model, operating system and thermal state reported by iOS. The application writes these results to a JSON file, which is then copied to the laptop. All timing figures in this paper come from these files.

IV. Results

A. Cross-dataset transfer

Table II shows poor transfer in both directions for the single-source models. The aggregate scores, however, combine two different failure mechanisms. On tomatOD, the Laboro model retains a recall of 0.656, compared with 0.879 for the tomatOD

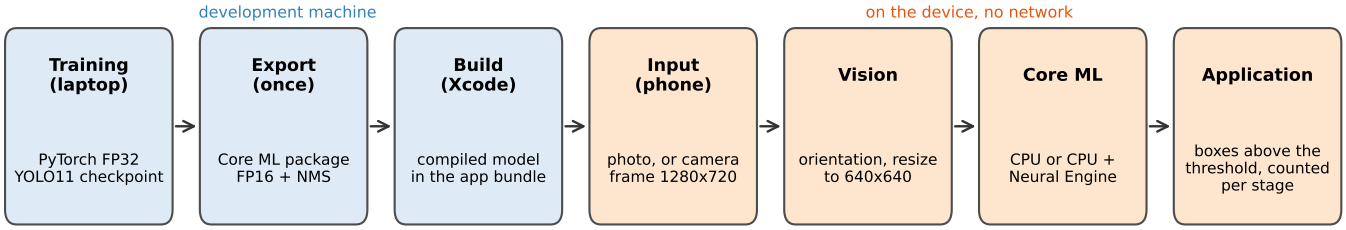


Fig. 2. From training to an on-device count. A PyTorch checkpoint is exported once to an FP16 Core ML package that already contains NMS; Xcode compiles it into the application; at run time the Vision framework resizes each photo or camera frame, Core ML runs the network on the CPU or on the Neural Engine, and the application counts the returned boxes per maturity stage.

TABLE II
MAP50–95 / COUNTING MAE AT CONFIDENCE 0.25. BOLD CELLS ARE IN-DOMAIN.

Model	Laboro validation	tomatOD validation
Laboro	**0.698 / 1.25**	0.389 / 3.29
tomatOD	0.308 / 2.14	**0.563 / 1.00**
Mixed	0.704 / 1.14	0.546 / 1.24

TABLE III
PER-STAGE COUNTS ON THE TOMATOD VALIDATION SPLIT AT CONFIDENCE 0.25. BIAS IS THE PREDICTED TOTAL AGAINST THE GROUND-TRUTH TOTAL. AN ASTERISK MARKS THE IN-DOMAIN MODEL.

Model	Stage	GT	predicted	bias
Laboro	green	292	501	+72%
	half-ripened	77	349	**+353%**
	fully ripened	99	49	**−51%**
tomatOD*	green	292	369	+26%
	half-ripened	77	106	+38%
	fully ripened	99	112	+13%
Mixed	green	292	398	+36%
	half-ripened	77	114	+48%
	fully ripened	99	135	+36%

specialist. It therefore finds many of the fruit but assigns them to the wrong stage. On Laboro, the tomatOD model falls to a recall of 0.484 and undercounts every stage. The tomatOD result includes a disagreement about maturity. The Laboro result is primarily a failure to find fruit.

Table III separates the stage counts. On tomatOD, the Laboro model predicts 4.5 times as many half-ripened fruit as are annotated and half as many fully ripe fruit. Its green count is also inflated, as are the counts from the other models at this threshold. The largest error is therefore a directed substitution between the half-ripened and fully ripe stages. On its own split, the tomatOD specialist over-predicts all three stages by between 13% and 38%. This consistent inflation across stages indicates a threshold effect. The in-domain Laboro result shows the same pattern (green +10%, half-ripened +20%, fully ripened +44%).

The colour measurement in Section III-C explains the direction of this error. In Laboro, the median background-corrected a^* values are -5.6 for green, 18.9 for half-ripened, and 41.7 for fully ripened fruit. In tomatOD, the corresponding values are -4.2, 6.6, and 13.6. The datasets therefore assign

TABLE IV
FITTED STAGE BOUNDARIES ON MEDIAN a^*, RAW AND AFTER SUBTRACTING THE IMAGE BACKGROUND MEDIAN. SELF-RECOVERY IS THE SHARE OF A DATASET'S OWN LABELS ITS OWN TWO CUTS REPRODUCE. THE SWAP ROWS APPLY ONE DATASET'S CUTS TO THE OTHER'S BOXES AND REPORT THE SHARE OF LABELS THAT CHANGE, OVERALL AND PER STAGE.

	raw a^*	background-corrected
Laboro, cuts	-10.1, +23.9	+3.2, +31.0
self-recovery	82%	76%
tomatOD, cuts	-3.4, +6.3	-0.1, +11.7
self-recovery	78%	78%
Laboro cuts on tomatOD	64%	22%
green / half / fully	71 / 1 / 97	1 / 25 / **98**
tomatOD cuts on Laboro	16%	27%
green / half / fully	1 / 77 / 9	20 / **75** / 7

similar colours to the green stage. TomatOD assigns the fully ripe label at a colour that Laboro assigns to the half-ripened stage. The gap between the datasets grows from 12 points for the half-ripened stage to 28 for the fully ripe stage. Figure 3 illustrates these values with real fruit.

The two fitted boundaries reproduce 76% to 78% of the labels in the dataset from which they were derived (Table IV). They therefore provide a usable description of each annotation convention. Applying the boundaries to the other dataset shows an asymmetric result. The Laboro boundaries place 98% of tomatOD's fully ripe boxes within Laboro's half-ripened band, matching the substitution observed in the transfer counts. In the other direction, the tomatOD boundaries move 75% of Laboro's half-ripened boxes into the fully ripe stage. The corresponding model-transfer result does not show this disagreement because the tomatOD model first fails to find many of the Laboro fruit.

The background correction separates the annotation difference from differences in image capture. In the raw a^* measurements, the two green medians differ by 10 points, -17.7 against -7.8. This difference is consistent with a change in illumination or white balance. Subtracting the background median of each image removes almost all of the green-stage gap. At the same time, it widens the gap between the half-ripened and fully ripe stages. The remaining disagreement after this per-image photometric normalisation is therefore associated with the annotation conventions.

The detector trained on both datasets retains the two an-

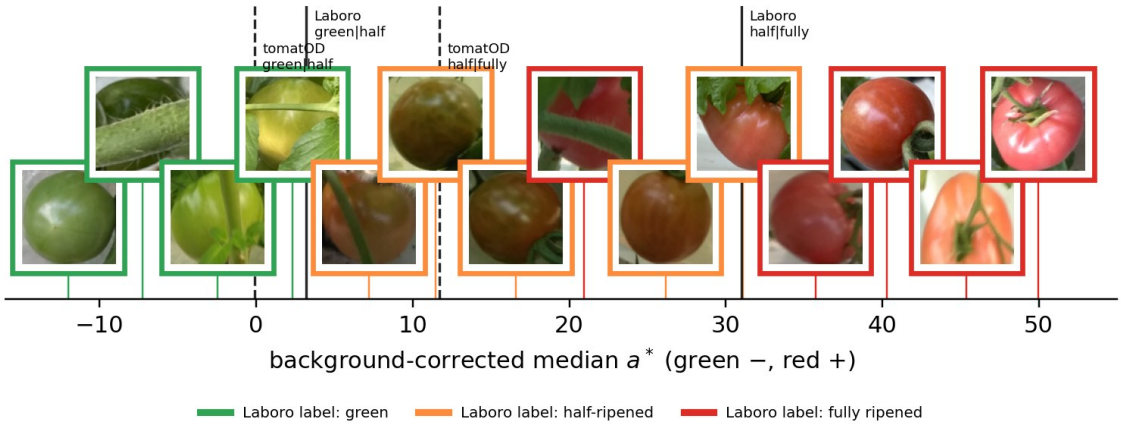


Fig. 3. Laboro validation fruit placed on the background-corrected a^* axis of Section III-C, framed in the colour of their Laboro label. Vertical lines are the stage boundaries fitted to each dataset (Table IV): solid for Laboro, dashed for tomatOD. A fruit at $a^* \approx 20$ is half-ripened by Laboro's boundaries and fully ripe by tomatOD's. Examples were chosen to span the axis and to contain no neighbouring annotated fruit; tomatOD images are not shown.

notation conventions. On Laboro, the mixed model matches the Laboro specialist (0.704 against 0.698 mAP50–95). On tomatOD, it is 0.017 below the tomatOD specialist. It also raises the two cross-dataset results from 0.308 to 0.704 and from 0.389 to 0.546. The mixed model does not reproduce the Laboro model's stage bias on tomatOD. The Laboro model gives a -51% fully ripe count, whereas the mixed model gives $+36\%$. This is the same direction as the tomatOD specialist, although Laboro supplies about 80% of the training boxes. Because the two source domains are visually distinguishable, the network can use a source-conditional maturity boundary. Section V considers the implication of this result for label noise.

The counting column in Table II has a limitation that Section IV-C measures. Those counts use a fixed confidence threshold of 0.25. Sweeping the threshold reduces the Laboro model's tomatOD counting error from 3.29 to 1.34. Thus, more than half of the apparent counting penalty under transfer is associated with the selected operating point, which is the confidence threshold used to produce the count. The mAP column is unaffected because mAP integrates over confidence.

Converting both collections to one taxonomy also revealed two defects in the published annotations. The tomatOD test split declares one category whose name contains a list of all three stage names, while its annotation records use three distinct category identifiers. Reading the categories separately for each split therefore collapses the test split onto one class without an error. The shipped annotations also contain 2,421 boxes, whereas the dataset documentation reports 2,418. These defects do not affect the results above because the experiments use the training and validation splits and map classes by name. A name-based mapping exposes both defects, while an identifier-based mapping would propagate them.

B. Occlusion separates detection from staging

All occlusion results in this subsection use the Laboro three-stage YOLO11n detector of Table II on the 161 Laboro validation images, which hold 1,996 annotated fruit. The ground-truth box is always the whole fruit, so recall measures whether the model found the fruit. Stage accuracy is computed only for the fruit that it found.

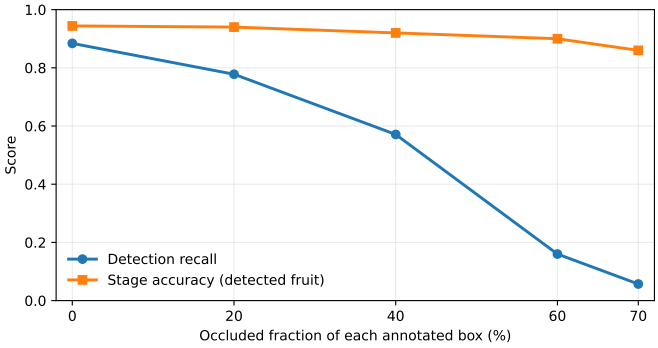


Fig. 4. Progressive synthetic occlusion. Detection recall collapses while maturity-stage accuracy among successfully detected fruit changes modestly.

Figure 4 shows that occlusion affects detection more than stage classification among detected fruit. Recall falls from 0.884 with nothing hidden to 0.057 at 70% hidden. Over the same range, stage accuracy for the detected fruit falls from 0.944 to 0.860, while counting MAE rises from 1.25 to 2.99. The mean confidence of matched detections also falls from 0.840 to 0.506, bringing the remaining detections closer to the 0.25 threshold. At 40% hidden, recall is 0.52 for green, 0.63 for half-ripened, and 0.71 for fully ripened fruit. Green fruit are the smallest fruit and are least distinct from the surrounding leaves. The stage with the lowest unoccluded recall therefore shows the largest loss.

Matching threshold. Recall drops sharply between 40% and 50% hidden. Part of this drop results from the metric definition. If a fraction f of a fruit is hidden and the model draws a box around only its visible part, that box has an IoU of about $1 - f$ with the whole-fruit ground truth. At $f = 0.5$,

TABLE V
RECALL UNDER SYNTHETIC OCCLUSION FOR ONE DETECTOR, SCORED FOUR WAYS. COLUMNS 2–3: SAME-IMAGE BACKGROUND STRIP, MATCHED AT IOU 0.5 AND 0.3. COLUMN 4: FLAT BLACK STRIP (IOU 0.5). COLUMN 5: THE MIXED-DATA DETECTOR WITH THE BACKGROUND STRIP (IOU 0.5). THE LAST TWO COLUMNS GIVE COUNTING MAE, WHICH DOES NOT DEPEND ON MATCHING.

	Recall				Counting MAE	
Hidden	IoU 0.5	IoU 0.3	black	mixed	backgr.	black
0%	0.884	0.895	0.884	0.878	1.25	1.25
10%	0.837	—	0.820	0.839	1.25	1.29
20%	0.778	—	0.709	0.770	1.33	1.43
30%	0.694	0.726	0.609	0.668	1.49	1.72
40%	0.571	0.642	0.461	0.554	1.77	1.88
50%	0.312	0.516	0.148	0.286	2.06	2.20
60%	0.160	0.344	0.091	0.136	2.43	2.53
70%	0.057	0.094	0.047	0.040	2.99	2.98

TABLE VI
NATURAL FRUIT-ON-FRUIT OVERLAP ON UNTOUCHED VALIDATION FRAMES: SHARE OF EACH ANNOTATED BOX COVERED BY OTHER ANNOTATED BOXES.

Covered	n	Recall	Stage acc.	Mean conf.
none	423	0.891	0.947	0.821
0–5%	149	0.913	0.949	0.865
5–15%	377	0.912	0.968	0.874
15–30%	503	0.913	0.939	0.860
30–50%	357	0.888	0.934	0.822
>50%	187	0.701	0.908	0.751

this value equals the usual matching threshold. A box that correctly describes the visible half is then counted as a miss against the whole-fruit annotation. Table V repeats the sweep with an IoU matching threshold of 0.3. The two thresholds agree to within 0.03 up to 30% hidden. At 50% hidden, they give 0.312 and 0.516. At 60%, they give 0.160 and 0.344. Counting MAE is identical under both thresholds because a count does not depend on the assignment between predictions and annotations. The IoU 0.5 column measures localisation of the whole fruit. The IoU 0.3 column measures whether the model detected any sufficiently localised part of it. Reporting both values distinguishes these two questions.

Occluder appearance. The black strip and the background strip cover exactly the same pixels. Differences between their columns therefore measure the detector's response to the strip appearance while the hidden area remains fixed. The black strip produces lower recall at every level from 10% upwards: 0.609 against 0.694 at 30% hidden, 0.461 against 0.571 at 40%, and 0.148 against 0.312 at 50%. The difference is about ten points of recall over the range that contains most natural occlusion. Synthetic occlusion results must therefore identify the occluder used. The mixed model, which is deployed in the application, remains within 0.03 of the Laboro detector at every level. The shape of the curve is therefore not specific to one checkpoint.

Natural overlap. Table VI groups the untouched validation fruit by the share of each box covered by other annotated boxes. Fruit with 30–50% box coverage retain a recall of 0.888, close to the 0.891 for isolated fruit. By comparison, a synthetic strip covering a nominal 30–40% gives recall of 0.571–0.694. Only above 50% box overlap does recall under natural overlap fall to 0.701. Two factors may explain this difference. Tomatoes are round and their boxes are rectangular, so boxes can overlap by a third even when neither fruit occludes the other. Box overlap can therefore overestimate the hidden fruit area. In addition, the training data contain fruit behind other fruit but do not contain the straight-edged strip. The synthetic-strip result is consequently a controlled stress test and does not estimate field recall.

TABLE VII
COLOUR-DIRECTED OCCLUSION AT 40% HIDDEN, PAIRED ON THE SAME FRUIT. MEAN PREDICTED STAGE ON A SCALE GREEN = 0, HALF = 1, FULLY = 2, FOR DETECTED FRUIT; "CHANGED" IS THE SHARE WHOSE STAGE DIFFERS BETWEEN THE TWO CONDITIONS. p: TWO-SIDED EXACT MCNEMAR TEST ON THE DIRECTION OF CHANGE.

GT stage	n	red hidden	green hidden	changed	p
green	493	0.010	0.018	1.8%	0.508
half-ripened	156	1.122	1.205	17.3%	0.019
fully ripened	187	1.941	1.957	4.8%	0.508

Which half is hidden. A half-ripened tomato is often red on one side and green on the other. Table VII covers either the redder or the greener half of the same fruit at 40% and compares the predicted stage. For green and fully ripe fruit, which have similar colours on either side, the prediction changes for 1.8% and 4.8% of fruit. For half-ripened fruit, it changes for 17.3%. Among the 27 half-ripened fruit with a changed prediction, 20 are classified as riper when the green side is covered and 7 when the red side is covered ($p = 0.019$). This result is consistent with the model using the visible colour to predict the stage, as suggested by the colour boundaries in Section IV-A. The effect occurs mainly in the stage whose colour varies across the fruit. The two directed modes are almost indistinguishable in the aggregate sweep. Their difference becomes visible in the paired comparison of the same fruit.

C. Confidence calibration changes counting conclusions

A detector produces a ranked list of boxes with confidence scores; a count is obtained by keeping every box above a threshold. The threshold therefore controls how many fruit are reported, even when the ranking of the boxes stays the same. This subsection examines how a fixed threshold can reverse a comparison between two training recipes.

Three YOLO11n recipes were trained on Laboro under the protocol of Table II and scored on tomatOD: the default recipe; the same recipe with the HSV colour jitter switched off (`no_colour`); and the default with CutMix and mixup added (`occlusion_aug`). At the usual threshold of 0.25 the cross-domain counting MAE is 3.29 for the default, 3.01 for `occlusion_aug`, and 1.50 for `no_colour`. At this fixed threshold, removing the colour jitter appears to halve the counting error on an unfamiliar dataset.

The results at 0.25 are chiefly explained by the number of boxes that each model returns. The default model predicts 1.92 times as many fruit as tomatOD contains, while the `no_colour` model predicts 0.93 times as many. Their

TABLE VIII
COUNTING ON TOMATOD BY THREE LABORO-TRAINED YOLO11N RECIPES, AT THE FIXED THRESHOLD OF 0.25 AND AT EACH MODEL'S BEST THRESHOLD FROM A SWEEP OVER 0.05–0.90. RATIO: PREDICTED OVER ANNOTATED FRUIT AT THE BEST THRESHOLD.

	MAE	At the best threshold			
Recipe	@0.25	MAE	conf.	recall	ratio
default	3.29	1.34	0.70	0.799	0.88
`no_colour`	1.50	1.50	0.25	0.746	0.93
`occlusion_aug`	3.01	1.32	0.70	0.776	0.85

stage accuracy on matched fruit differs by less than 0.006. The models therefore assign similar stages to matched fruit but place their confidence scores differently relative to the threshold. This difference is a property of calibration.

Table VIII sweeps the threshold. The default model reaches 1.34 at confidence 0.70 and keeps a recall of 0.799; the `no_colour` model is already at its best at 0.25, with 1.50 and a recall of 0.746. Once each model uses its best operating point, meaning the selected confidence threshold and the resulting recall, the ordering reverses. The colour-jittered model has lower counting error on the unfamiliar dataset and retains more recall. The `no_colour` model appeared better because its calibration was closer to the fixed threshold. The mAP result does not change because mAP averages over every threshold.

Threshold selection also affects the transfer results in Section IV-A. The Laboro model's counting MAE on tomatOD falls from 3.29 at 0.25 to 1.34 at its best threshold. Threshold selection therefore accounts for more than half of the apparent transfer penalty in Table II; the change of dataset does not account for all of it. On Laboro, the sweep retains a smaller advantage for `no_colour` (0.93 against 1.03). This agrees with its 0.008 higher mAP50–95, but the comparison uses a single seed and is not treated as a finding.

Figure 5 shows the same mechanism in-domain, across the model-size ladder of Section IV-E. At 0.25 every size predicts between 1.12 and 1.26 times as many fruit as are annotated, and its counting MAE is 0.15–0.46 above its own minimum. The error curves are flat between roughly 0.4 and 0.65 and rise steeply on either side. Their minima lie at thresholds from 0.40 (YOLO11n) to 0.60 (YOLO11l and x). A single fixed threshold therefore favours some sizes over others (Table XI).

CutMix and mixup were added to test whether pasting image regions over the fruit during training makes the detector more robust to the occlusion of Section IV-B. The results do not show greater robustness. Under the background strip, the augmented model reaches recall 0.671 against the default's 0.694 at 30% hidden, and 0.283 against 0.312 at 50%. Its small advantage in tuned counting on tomatOD (1.32 against 1.34) is well within single-seed noise. Rectangular occlusion during training therefore provides no evidence of robustness to either the test intervention or natural leaves.

D. Core ML and smartphone performance

In the laptop evaluation, the FP16 Core ML export changes recall by at most 0.002 and counting MAE by 0.01 relative to FP32 PyTorch. The iPhone Neural Engine runs the same package at a median 3.8 ms per frame (Table IX). All 30 detections in the bundled samples match between the Mac and the iPhone, with identical labels and a mean box IoU from 0.992 to 0.997 across the compute-unit settings. Compute units specify the processors used for inference. Their outputs are not bit-identical: one borderline fruit changes label between the CPU and Neural Engine settings.

TABLE IX
RUNTIME COMPARISON FOR THE MIXED YOLO11N DETECTOR.

Runtime	Precision	ms/frame	FPS
Mac PyTorch MPS	FP32	28.1	36
Mac Core ML CPU	FP16	16.4	61
Mac Core ML CPU+ANE	FP16	6.0	166
iPhone CPU	FP16	11.7	86
iPhone CPU+ANE	FP16	3.8	262

TABLE X
THE YOLO11 LADDER TRAINED UNDER ONE PROTOCOL ON LABORO. COUNTING MAE IS THE BEST VALUE OVER THE CONFIDENCE SWEEP, REPORTED FOR ALL STAGES AND FOR FULLY RIPE FRUIT ALONE, WITH THE THRESHOLD THAT ACHIEVES IT; TABLE XI GIVES THE SAME QUANTITY AT A FIXED THRESHOLD. PACKAGE SIZE IS THE FP16 CORE ML ARTEFACT; LOAD TIME IS MEASURED ON THE PHONE.

Model	Params	GFLOPs	mAP	MAE	ripe MAE	MB
YOLO11n	2.58 M	6.3	0.706	0.963	**0.590**	5.4
YOLO11s	9.43 M	21.3	0.747	0.919	0.640	19.1
YOLO11m	20.03 M	67.7	**0.782**	0.882	0.627	40.3
YOLO11l	25.32 M	86.6	0.780	**0.859**	0.658	51.0
YOLO11x	56.88 M	194.4	0.774	0.874	0.602	114.1

E. Effect of model size on accuracy and cost

mAP50–95 rises from 0.706 for n to 0.747 for s and 0.782 for m. YOLO11l is 0.002 below m, and YOLO11x is 0.008 below m. Both differences are within the variation that can be distinguished from a single seed (Table X). Tuned counting error continues to fall to 0.859 for l. However, one seed does not support an ordering among m, l, and x.

Fully-ripe counting MAE does not decrease with model size. The best value on the ladder is 0.590 for YOLO11n. Every larger model gives a value between 0.602 and 0.658. Higher overall detection accuracy therefore does not imply lower fully-ripe counting error. A stage count depends first on detecting a fruit and then on assigning its maturity stage. These results suggest that additional capacity improved detection without improving the stage assignment.

From m to x, the Core ML package grows 2.8×, Neural Engine latency grows 2.0×, and model load time grows 1.7×. The mAP does not increase (Figure 6).

The counting column of Table X uses a threshold selected separately for each model. That threshold is selected on the same 161 images used for the comparison. Table XI reports two operating points, meaning the selected confidence threshold and the predictions retained at that threshold. At the fixed confidence of 0.25, counting MAE does not improve with capacity. The smallest model has the lowest overall MAE at 1.112, YOLO11m has the highest at 1.346, and the ladder

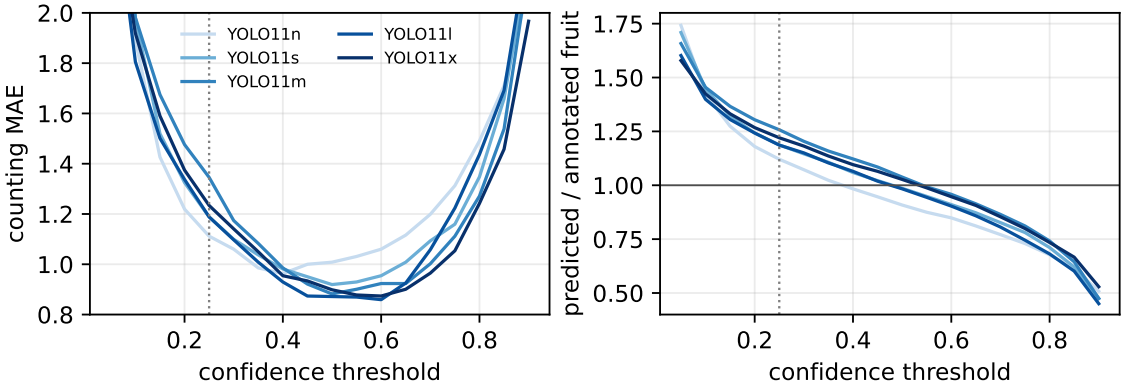


Fig. 5. Counting MAE (left) and the ratio of predicted to annotated fruit (right) against the confidence threshold, for the five YOLO11 sizes on the Laboro validation split. The dotted line marks the default threshold of 0.25. Every size over-counts at 0.25, and the threshold at which the ratio crosses 1.0 differs between sizes.

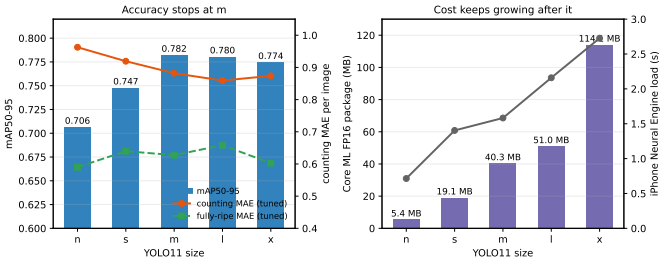


Fig. 6. mAP50–95, tuned counting MAE, Core ML package size and model load time across the five YOLO11 sizes.

TABLE XI
COUNTING MAE AT A FIXED CONFIDENCE OF 0.25 AND AT THE PER-MODEL BEST THRESHOLD, BOTH READ FROM THE SAME SWEEP OVER THE SAME 161 VALIDATION IMAGES. THE BEST-THRESHOLD COLUMNS GIVE THE ACHIEVING CONFIDENCE IN BRACKETS.

	all stages		fully ripe	
Model	0.25	best	0.25	best
YOLO11n	**1.112**	0.963 (0.40)	0.820	**0.590** (0.60)
YOLO11s	1.188	0.919 (0.50)	0.801	0.640 (0.60)
YOLO11m	1.346	0.882 (0.50)	1.000	0.627 (0.60)
YOLO11l	1.188	**0.859** (0.60)	**0.758**	0.658 (0.40)
YOLO11x	1.234	0.874 (0.60)	0.839	0.602 (0.60)

spans 0.234 without an ordering by size. Threshold tuning changes each model's MAE by between 0.149 and 0.464. This change exceeds the 0.104 difference across the entire tuned ladder. The monotone decrease to 0.859 at YOLO11l therefore results from selecting a threshold for each model and does not establish an effect of capacity. The tuned figures show the best counting result each model reaches on these images; they do not provide an unbiased comparison between models.

Fully-ripe MAE has no ordering by model size at either operating point. The best tuned value is YOLO11n's 0.590, while the best value at 0.25 is YOLO11l's 0.758. YOLO11m is the worst of the five models in the fixed-threshold column. Thus, capacity does not improve the fully-ripe count on this split under either comparison. Section V uses this result.

TABLE XII
MEDIAN INFERENCE LATENCY ON BOTH DEVICES, FPS IN BRACKETS, FP16 CORE ML ON IDENTICAL IMAGES. THE LAST COLUMN IS THE NEURAL ENGINE SPEED-UP OVER THE PHONE'S OWN CPU.

	CPU only		CPU + Neural Engine	
Model	Mac	iPhone	Mac	iPhone
YOLO11n	9.1 (110)	13.6 (74)	1.76 (570)	6.76 (148)
YOLO11s	17.0 (59)	26.1 (38)	3.25 (307)	8.20 (122)
YOLO11m	32.4 (31)	49.0 (20)	9.48 (106)	13.72 (73)
YOLO11l	42.2 (24)	59.0 (17)	11.11 (90)	15.22 (66)
YOLO11x	69.1 (14)	112.0 (9)	22.51 (44)	27.21 (37)

F. Laptop and smartphone measurements are not interchangeable

The ratio between laptop and smartphone latency varies with model size (Table XII, Figure 9). On the CPU, the smartphone latency is 1.40 to 1.62 times the laptop latency at every size. On the Neural Engine, the ratio is 3.85× for YOLO11n and falls monotonically to 1.21× for YOLO11x. The Neural Engine speed-up over the smartphone's own CPU increases with model size, from 2.0× for n to 4.1× for x.

This pattern is consistent with a fixed per-inference overhead on the smartphone, although the benchmark does not isolate that overhead directly. A fixed overhead would have the largest proportional effect on the smallest model. Regardless of its cause, the measured laptop latency understates the smartphone latency most strongly for the small models considered for mobile deployment.

Accuracy stops improving at m, while the real-time limit occurs three sizes later. YOLO11x only narrowly meets the smartphone budget: 36.7 FPS in a short run, 33.8 FPS sustained over 4,500 frames, and a p90 of 31.8 ms against a 33.3 ms budget (Table XIII). Its sustained median is 8.8% above its short-run median, the largest drift in the ladder. YOLO11l drifts 2.7%, and the reported thermal state remains *fair* or better throughout. A warmer smartphone, a larger input, or processing outside inference would cause YOLO11x to exceed the budget.

If the Neural Engine is unavailable and the application

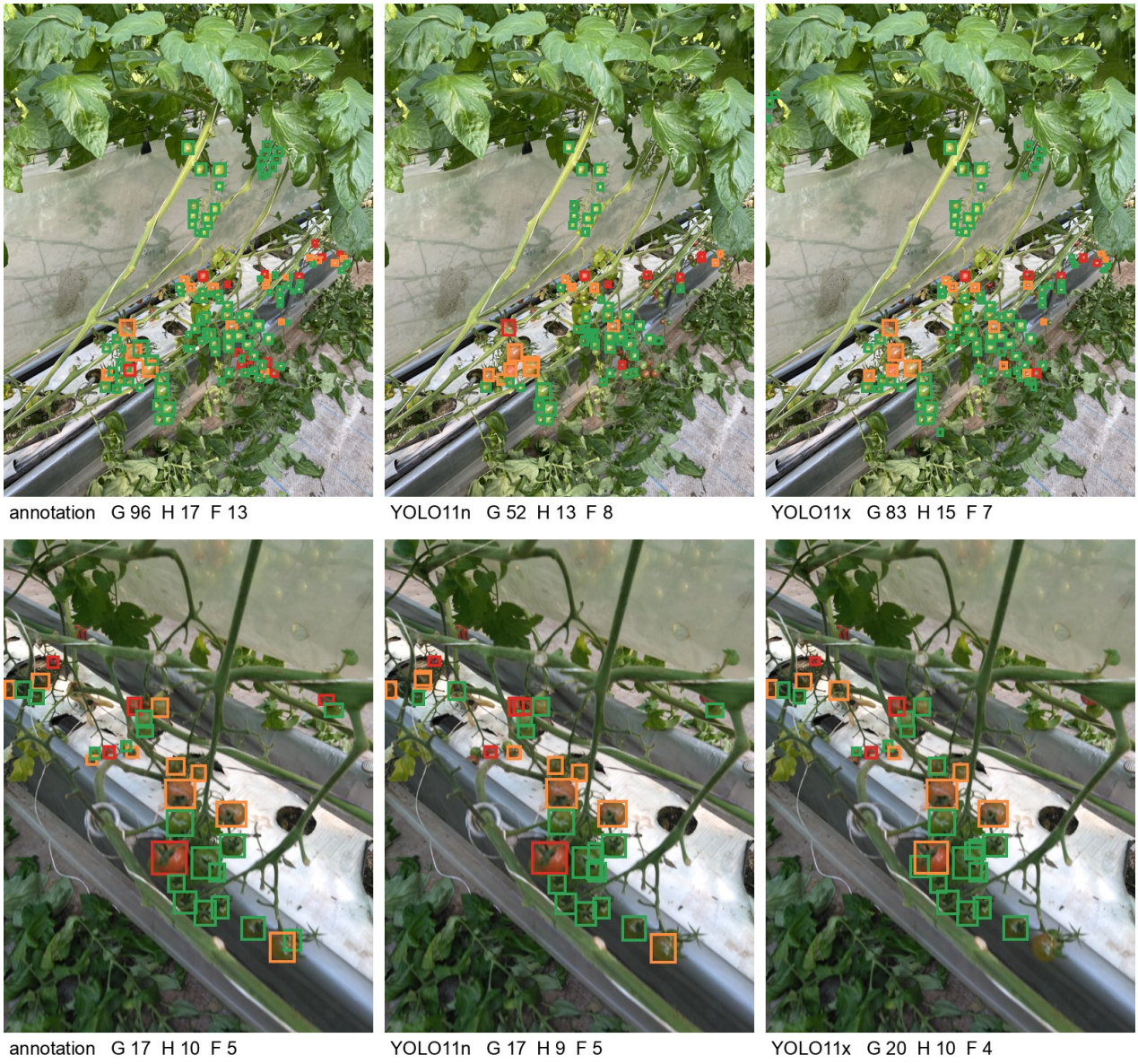


Fig. 7. Annotation (left), YOLO11n (centre) and YOLO11x (right) on the two Laboro validation frames with the most half-ripened fruit, at confidence 0.25. Box colour is the maturity stage (green, orange for half-ripened, red for fully ripened); the counts of green (G), half-ripened (H) and fully ripened (F) fruit are printed under each panel. In the upper frame, which is dense with small fruit, YOLO11x finds far more of the green fruit than YOLO11n (83 against 52 of 96). In the lower frame both models count almost exactly, but YOLO11x labels the large red fruit in the centre half-ripened, while YOLO11n agrees with the annotation. The frames were selected automatically and are illustrations, not additional results.

falls back to the CPU, as the shipped build did because of an unrelated compiler assertion on the GPU path, the larger models fail to meet the budget three sizes earlier. YOLO11s is the last size above 30 FPS at 38.3; m runs at 20.4, and x runs at 8.9.

Cross-device outputs agree closely. Across all five sizes every matched box has a mean IoU between 0.990 and 1.000 with its laptop counterpart and an identical stage label. Two borderline boxes out of roughly thirty differ in count, which is the same order of disagreement observed between compute units on one device. The latency differences are therefore unlikely to result from different detections.

Between the two measurement days the iPhone was updated from iOS 26.5.2 to iOS 27.0. YOLO11n's Neural Engine latency changed from 4.05 ms to 6.4–6.8 ms across that update, repeatably, while YOLO11m was unchanged at 13.5–13.7 ms and CPU-only latency was unchanged for both. A 67% regression for the smallest model, with no corresponding change for a larger model, is further evidence of a fixed overhead instead of a change in compute throughput. An on-device latency result therefore requires the OS version for interpretation.

G. *Response to lighting changes*

Table XIV compares model size and lighting changes using the same recall scale. Moving from YOLO11n to YOLO11x raises recall on clean frames by 0.058. Over-exposing the same frames by three stops reduces recall for the best model in the ladder by 0.205.

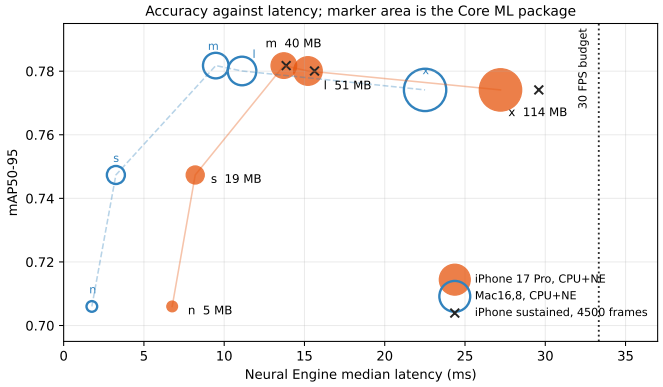


Fig. 8. mAP50–95 against Neural Engine latency on both devices. Marker area is the Core ML package size; crosses mark the sustained runs.

TABLE XIII
EACH SIZE AGAINST A 30 FPS CAMERA BUDGET ON THE PHONE'S NEURAL ENGINE. THE SUSTAINED COLUMN IS A 4,500-FRAME RUN; YOLO11M'S SUSTAINED FIGURE WAS COLLECTED ON THE EARLIER OS VERSION AND IS NOT COMPARABLE WITH ITS OWN SHORT RUN. YOLO11N AND S WERE NOT RUN SUSTAINED (N/R), SINCE THEIR SHORT-RUN P90 ALREADY LEAVES A MARGIN OF MORE THAN THREE TIMES THE BUDGET.

Model	median	p90	budget/p90	sustained
YOLO11n	6.76	7.91	4.22×	n/r
YOLO11s	8.20	10.52	3.17×	n/r
YOLO11m	13.72	15.32	2.18×	13.86 (72.2 FPS)
YOLO11l	15.22	16.68	2.00×	15.63 (64.0 FPS)
YOLO11x	27.21	28.30	1.18×	29.60 (33.8 FPS)

The response differs between the two exposure directions. At −3 stops (mean luma 42 out of 255), recall falls by at most 0.03 for any model. It does not fall for the variants with the widest jitter. At +3 stops, recall drops by 0.15 to 0.21 and counting MAE roughly doubles. The examples in Figure 10 indicate a possible explanation. Darkening compresses the signal while preserving the ratios between channels. Clipping removes those ratios, and an over-exposed red tomato loses the saturation used to assign its stage. Additional capacity does not recover this information. YOLO11x performs no better at +3 stops than YOLO11s.

The two white-balance directions also produce different results. A cool cast reduces recall more than a warm cast of the same magnitude. A warm cast has a larger effect on counting than on detection. It moves green and half-ripened fruit along the green-to-red axis towards riper predictions, causing the ripe stage to be over-counted. The annotation disagreement measured in Section III-C also shifts the apparent colour boundary along this axis. The camera causes the shift in the lighting test, while the annotators cause it in the cross-dataset test.

H. Effect of wider colour augmentation

This section tests whether stronger photometric augmentation improves robustness to the lighting interventions. We first measure variation across seeds because it is similar in magnitude to the effects being compared.

TABLE XIV
RECALL AT CONFIDENCE 0.25 AND COUNTING MAE AT THE BEST THRESHOLD UNDER THE FOUR EXTREME LIGHTING CONDITIONS. ONE SEED PER ROW; THE COLOUR-AUGMENTATION ROWS ARE REPEATED OVER THREE SEEDS IN TABLE XV.

Model	clean	−3 st.	+3 st.	cool	warm
YOLO11n	0.877	0.860	0.696	0.797	0.817
no colour jitter	0.877	0.765	0.608	0.751	0.782
hsv_v08	0.874	0.870	0.692	0.810	0.820
hsv_wide	0.871	0.868	0.704	0.824	0.847
YOLO11s	0.916	0.904	0.739	0.842	0.883
YOLO11m	0.935	0.901	0.786	0.870	0.898
YOLO11l	0.929	0.912	0.782	0.877	0.901
YOLO11x	0.935	0.914	0.730	0.865	0.899
counting MAE at the best threshold					
YOLO11n	0.96	1.13	1.78	1.31	1.35
no colour jitter	1.00	1.47	2.07	1.52	1.71
YOLO11m	0.88	1.05	1.68	1.11	1.27
YOLO11x	0.87	0.93	1.81	1.23	1.06

TABLE XV
COLOUR-AUGMENTATION RECIPES OVER THREE SEEDS: RECALL AT CONFIDENCE 0.25, MEAN (OBSERVED RANGE). THE SPREAD OF THE DEFAULT RECIPE AGAINST ITSELF IS THE NOISE FLOOR FOR THIS COMPARISON.

Recipe	clean	−3 st.	+3 st.	cool
default	0.875 (0.874–0.877)	0.859 (0.857–0.860)	0.666 (0.647–0.696)	0.792 (0.773–0.807)
`hsv_v08`	0.872 (0.868–0.875)	**0.872** (0.870–0.874)	**0.707** (0.692–0.728)	0.794 (0.774–0.810)
`hsv_wide`	0.874 (0.871–0.880)	0.870 (0.862–0.881)	0.694 (0.686–0.704)	**0.808** (0.781–0.824)
`no_colour`	0.876 (0.872–0.879)	0.757 (0.739–0.768)	0.604 (0.567–0.636)	0.758 (0.731–0.791)

The spread across seeds depends on the lighting level. Retraining the *same* default recipe at three seeds changes clean recall by 0.003. At +3 stops, recall changes by 0.049, from 0.647 to 0.696 (Table XV). The single-seed value of 0.696 for the default at +3 stops in Table XIV is the most favourable of its three runs. A conclusion based on one run at this extreme level would therefore be within the observed retraining variation.

Removing colour jitter has no detectable effect on clean frames but a large effect under perturbation. On clean frames, `no_colour` is indistinguishable from the default. Its recall is +0.001, and its counting error is 0.043 *lower* with an interval that spans zero ($p = 0.16$). Under every perturbation, it gives lower recall: −0.102 at −3 stops, −0.062 at +3, and −0.034 and −0.036 under the two colour casts. Its counting error is also worse by 0.17 to 0.32, with Holm-adjusted $p \leq 0.006$ at each of the four levels (Table XVI). An ablation evaluated only on clean held-out frames would therefore recommend removing an augmentation that improves every perturbed condition tested here.

Widening the jitter produces smaller, perturbation-specific gains. Doubling the brightness half-width improves the two exposure extremes. At +3 stops, the changes are −0.136 counting error and +0.041 recall. At −3 stops, they are −0.075 and +0.013. Both results are significant after correction, while neither white-balance level differs from the

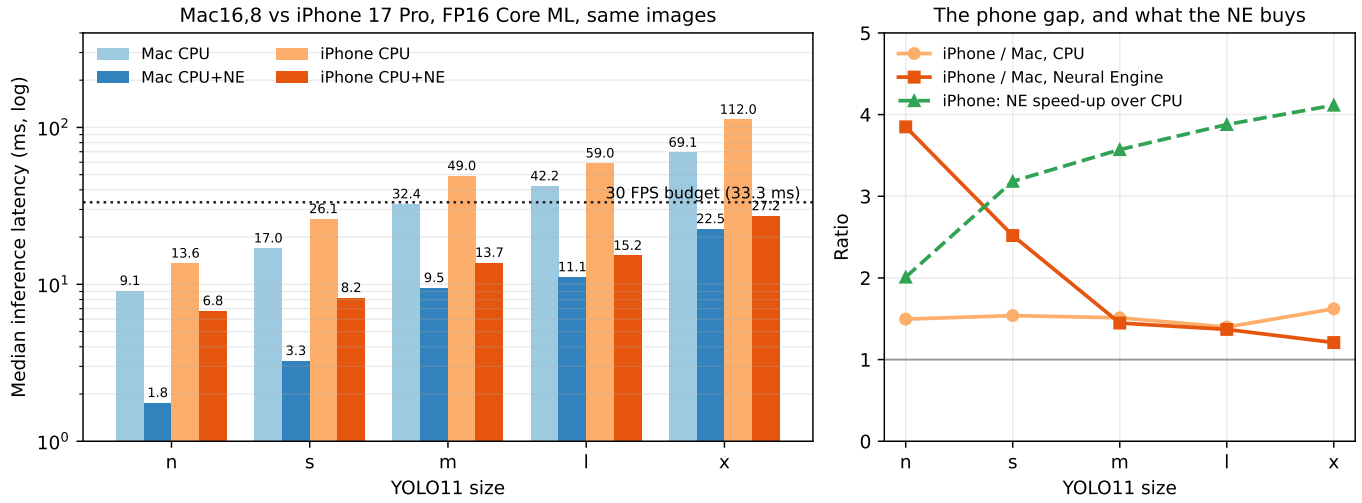


Fig. 9. Inference latency for every size on both devices and both compute unit settings, against a 33.3 ms frame budget (left), and the resulting ratios (right). The laptop-to-phone gap is flat on the CPU and closes with model size on the Neural Engine.



Fig. 10. One validation frame under the photometric interventions: exposure from -3 to $+3$ stops (top) and white balance from cool to warm (bottom). Captions give the mean luma of the frame and the baseline detector's recall over the whole validation split at that level. Annotations are never moved; only the pixels change.

default. Widening hue produces its clearest changes under the white-balance intervention. At a cool cast, the changes are -0.168 counting error ($p = 0.002$) and $+0.016$ recall. Recall changes by $+0.021$ at a warm cast. Its exposure changes remain within the interval, and clean-frame counting error increases by $+0.062$; this increase is not significant after correction.

Each modified component improves the corresponding type of perturbation. Increasing the brightness half-width improves the two exposure conditions. Widening hue, the HSV change most similar to a colour cast, improves the two white-balance conditions. Neither modification produces an improvement across all photometric conditions.

The effect sizes support retaining the default recipe. Removing the jitter reduces recall by up to 0.102. Widening it improves recall by at most 0.041 and only for the corresponding perturbation. If a deployment is known to encounter one type of change, such as harsh backlight, widening the relevant half-width provides a small but measurable improvement.

These conclusions rest on three seeds per recipe and on paired tests over 161 images. The paired intervals quantify uncertainty on this validation set. The seed ranges show that a single retraining would not reliably reproduce the smaller effects, which are below about 0.02 recall.

V. Discussion

Several protocol choices changed the reported error while the trained weights remained fixed. In the dataset-transfer experiment, the annotation convention changed with the dataset. The synthetic-occlusion result depended partly on the geometry and appearance of the occluder. A count produced at a fixed

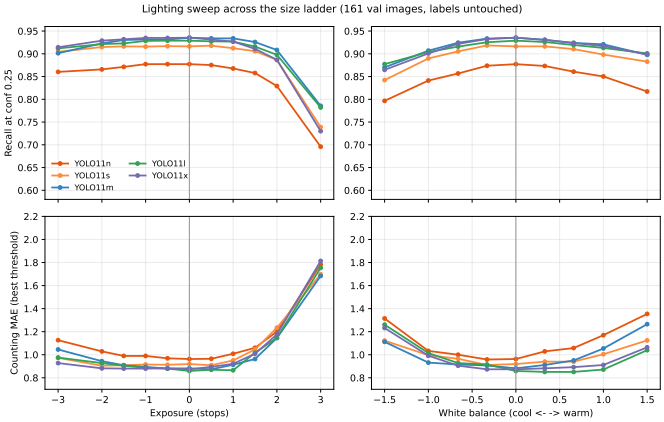


Fig. 11. Recall at a fixed threshold and counting error at the best threshold along the exposure and white-balance axes for the whole size ladder. The curves are ordered by capacity on clean frames and converge at the extremes.

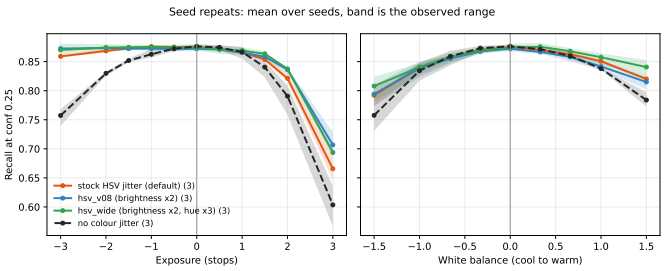


Fig. 12. Three seeds per training recipe. The line is the mean over seeds and the band is the observed range. Removing colour jitter separates from the three jittered recipes only once the frame is perturbed, and the seed spread itself widens with the perturbation.

threshold also reflected the calibration of the model's confidence scores. The photometric sweep measured the response to simulated camera settings. The deployment comparison included changes in precision, NMS, padding, silicon, and operating-system version. The metrics remain useful when the protocol specifies these factors and reports the relevant controls and complementary measurements.

A. Capacity has the smallest measured effect

The measured effect sizes differ substantially. Moving from the smallest to the largest model on this split changes clean recall by 0.058 and mAP50–95 by 0.068. The improvement ends at the medium model, two sizes below the largest model tested. Over-exposing the same frames by three stops reduces recall by 0.15 to 0.21, while covering 40% of each fruit reduces it by more than 0.3. Changing the annotation convention changes the assigned stage for a large fraction of correctly detected half-ripened fruit. Selecting a different confidence threshold changes cross-domain counting error by a factor of two without changing mAP.

In this study, threshold calibration and exposure control changed recall or counting error more than model scaling. Model capacity is nevertheless the variable most often optimised. This comparison concerns measured effect sizes; the study did not measure the implementation cost of the interventions.

TABLE XVI

EACH RECIPE AGAINST THE DEFAULT, SEED-AVERAGED AND PAIRED OVER THE 161 VALIDATION IMAGES. ΔMAE IS THE CHANGE IN COUNTING ERROR, NEGATIVE FAVOURING THE RECIPE; ΔRECALL IS AT CONFIDENCE 0.25. BRACKETS ARE 95% BOOTSTRAP INTERVALS AND p IS HOLM-ADJUSTED OVER THE FIVE LEVELS.

Recipe	Level	ΔMAE [95% CI]	Δrecall
`hsv_v08`	clean	$+0.005$ $[-0.039, +0.049]$	-0.003
	-3 st.	-0.075 $[-0.130, -0.022]^{*}$	$+0.013$
	$+3$ st.	-0.136 $[-0.260, -0.039]^{*}$	$+0.041$
	cool	-0.062 $[-0.133, +0.008]$	$+0.002$
	warm	$+0.039$ $[-0.021, +0.101]$	-0.005
`hsv_wide`	clean	$+0.062$ $[+0.007, +0.116]$	-0.001
	-3 st.	-0.035 $[-0.101, +0.029]$	$+0.011$
	$+3$ st.	-0.045 $[-0.138, +0.039]$	$+0.028$
	cool	-0.168 $[-0.261, -0.075]^{**}$	$+0.016$
	warm	-0.001 $[-0.108, +0.093]$	$+0.021$
`no_colour`	clean	-0.043 $[-0.103, +0.017]$	$+0.001$
	-3 st.	$+0.324$ $[+0.208, +0.450]^{***}$	-0.102
	$+3$ st.	$+0.165$ $[+0.058, +0.280]^{**}$	-0.062
	cool	$+0.203$ $[+0.115, +0.295]^{***}$	-0.034
	warm	$+0.226$ $[+0.100, +0.364]^{**}$	-0.036

$^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$, Holm-adjusted

B. What the fully-ripe count says about capacity

Detection improves across part of the model-size series, but fully-ripe counting error does not. The smallest model has the best value (Section IV-E). A stage count requires the detector to find the fruit and then assign a maturity stage from the colour within the detected box. The stage assignment is affected by the annotation boundary in Section III-C and the illumination in Section IV-G. Increasing the network's depth and width does not address either factor. Figure 7 shows an example: on the same frame the largest model labels a clearly red fruit half-ripened while the smallest model agrees with the annotation. Fully-ripe MAE should therefore be reported alongside mAP.

C. Latency depends on the deployment device

The laptop-to-smartphone latency ratio falls from 3.85× for the smallest model to 1.21× for the largest. A development-machine benchmark therefore underestimates smartphone latency most strongly for the small models considered for mobile deployment. The measurement also changes over time. An operating-system update changed the smallest model's accelerator latency by 67% while leaving a larger model unchanged. Reproducing a latency result requires the device, compute-unit setting, which specifies the permitted processor combination, OS version, and run duration.

D. Augmentation ablations need an out-of-distribution test

A clean-only ablation would favour removing the colour jitter. Removing it has no measured penalty on clean frames and gives the best clean mAP among the four recipes. Under a perturbation generated by a different process, however, the model trained without colour jitter loses up to 0.095 recall. Claims about augmentation concern behaviour outside the training distribution and should be evaluated under such

conditions. The test intervention should also differ from the augmentation process used in training.

The mixed-data result requires a related qualification. The two source domains are visually distinguishable, so the network can learn a different maturity boundary for each source. Random label flips provide no corresponding visual cue. This distinction motivates a broader study comparing label noise whose source can be inferred from the image with noise whose source cannot be inferred.

The occlusion study shows the value of testing several levels of severity, consistent with prior controlled benchmarks [41]. It also confirms the need for multiple occluders and natural controls. The gap between the background and black patches shows the effect of occluder appearance. The smaller loss under untouched fruit overlap also agrees with evidence that robustness to artificial occlusion need not transfer to realistic occlusion [42].

E. Comparison with previous work

Previous tomato maturity detectors report mAP50 values between about 0.86 and 0.97 [12], [13], [14], [15], each on its own dataset, with its own class definitions and its own train/test split. These differences in evaluation conditions prevent a direct comparison with one another or with the values reported here. In Table II, the same YOLO11n detector, trained once, scores 0.698 mAP50–95 on the dataset it was trained on and 0.389 on the other public tomato dataset, which uses different annotators and a different colour boundary for the ripe stages. A difference of a few points between two published detectors is much smaller than the difference produced by changing the test set alone.

The datasets also have different policies for unlabelled fruit. tomatOD leaves fruit that are more than half hidden unannotated [8], so a detector evaluated on it is never asked to find heavily occluded fruit, whereas Section IV-B shows that recall falls steeply above 50% hidden. Studies that target occluded fruit directly, such as Mu *et al.* for green tomatoes [26], therefore measure a different task from studies that report accuracy on tomatOD, even when both report mAP.

On lighting, the results agree with Haggag *et al.*, who found illumination the hardest of the shifts they tested for a greenhouse tomato detector [39]. Here, three stops of overexposure reduced recall by more than three times as much as the whole difference between the smallest and the largest model. Unlike that study, this work also tested whether the colour augmentation already in the training pipeline explains the measured robustness, and repeated the comparison over three seeds.

On deployment, earlier tomato studies on smartphones and embedded boards report the speed of one model on one platform [17], [18], [19], [20]. The model-size series in Section IV-E shows that the ratio between laptop and smartphone latency changes from 3.85 to 1.21 across model sizes, and that an operating-system update changed the latency of one model by 67% while leaving another unchanged. A single speed figure from another device, model size or operating-system version is therefore a weak guide to the speed of a different configuration. A deployment study needs to report all four: the device, model size, compute-unit setting and operating-system version.

VI. Reproducibility

The tables and figures in this paper are generated from saved outputs rather than transcribed by hand. Training runs write their own metric history, each sweep writes a JSON record of every level and threshold, and the smartphone benchmark writes a report that is pulled from the device. Non-regenerable measurements, particularly smartphone reports, are stored under unique filenames and never overwritten. The one such loss that occurred during this work is reported beside the measurements it affects.

The companion iOS application bundles the exported ladder, exposes the model and compute-unit selection used in Section IV-E, and runs the benchmark unattended. The reported device measurements therefore come from the same build that a user would install.

VII. Limitations

Training results outside Section IV-H use one seed, so differences of the order of 0.01 mAP50–95 or 0.02 recall are reported as indistinguishable and are not ranked. The colour-augmentation comparison is repeated across three seeds because its effect sizes are of that order. The small gains from widening the jitter remain close to the spread between retrainings, even where the paired test over images distinguishes them. Three seeds provide only a loose bound on that spread. They are enough to show that removing the jitter costs an order of magnitude more than widening it returns. They are not enough to rank the two widened recipes against each other.

Counting error at a tuned threshold is optimistic because each model's confidence threshold is selected on the same 161 validation images used to report the comparison. There is no separate split for threshold selection. The tuned columns in Tables X and XI therefore show what each model can reach on these images, not what it would necessarily reach on unseen frames. The fixed-threshold columns are reported alongside the tuned results. Section V uses only conclusions that hold at both operating points.

Both principal datasets are public research-greenhouse collections, and no held-out field collection has yet been scored. The photometric sweep simulates changed light on the existing frames. It does not use images recaptured under different lighting and cannot reproduce the sensor noise, motion blur, or specular behaviour of a genuinely dark or genuinely overexposed image. Natural occlusion is estimated from box overlap; instance masks would give a better area estimate but would still not establish depth order. The synthetic strip has an unnatural straight boundary and simultaneously occludes all annotated fruit in each frame.

The smartphone timing experiment uses three bundled samples, so it measures model inference and not end-to-end live-camera throughput. The GPU compute path is excluded because it failed to compile on the device. The size ladder is trained only on Laboro and is not directly comparable with the

mixed model used in the deployment table. It is also trained at one input resolution, leaving the trade-off between accuracy, latency, and input size unmeasured. Power is not reported because the automated Power Profiler runs ended unreliably and did not yield a defensible value [55].

VIII. Conclusion

This work asked how much of a reported tomato counting error belongs to the measurement protocol and how much to the detector. Six protocol choices were varied one at a time, and the training recipe was held fixed wherever the question allowed it. The choices were the annotation convention that defines a maturity stage, the occluder, the confidence threshold, the illumination of the test frames, the model size, and the runtime used to execute the model.

The measured effects differ in magnitude, and model size does not produce the largest change. Across the YOLO11 ladder trained under one protocol, clean recall spans 0.058 and mAP50–95 spans 0.068. Both stop improving at YOLO11m, while package size, latency, and load time continue to grow through YOLO11x. Over-exposing the same validation frames by three stops reduces the strongest model's recall by 0.205. A cool white-balance cast reduces recall by 0.05 to 0.08. Hiding 40% of each fruit reduces recall by more than 0.3. This result also depends on the occluder's appearance: recall is 0.571 under a same-image background patch and 0.461 under a black patch at the same hidden fraction. Changing the annotation convention places 98% of one dataset's fully ripe boxes within the other dataset's half-ripened band. The class-level transfer error follows the same direction. Changing the confidence threshold alters cross-domain counting error by more than a factor of two while leaving mAP unchanged. Selecting the threshold separately for each model also reorders the counting results in the size ladder. The FP16 Core ML export changes recall by at most 0.002. For the same package, latency varies with the compute units (2.0× to 4.1×), the device (1.21× to 3.85× against a laptop), the operating-system version (67% for the smallest model across a single update), and the run length (8.8% drift over 4,500 frames for the largest).

A protocol based on one summary number would have changed three conclusions. An ablation evaluated on clean held-out frames would have recommended removing the colour jitter. Removing it has no measured cost on clean data but reduces recall by up to 0.102 after perturbation. A single training run at the most perturbed level reported the default recipe at the most favourable of its three seeds. The tuned counting column of the size ladder implies that counting improves with capacity, while the fixed-threshold column does not support that conclusion.

These results establish a reporting requirement. A reproducible counting error must identify the annotation source and the colour boundary used by that source. An occlusion result must identify the occluder and matching rule. A counting result must report the confidence threshold and the split used to select it, while a lighting evaluation must state its illumination range. A latency result must identify the device, compute units, operating system, and run duration. Reporting these details separately allows the detection, staging, counting, and deployment results to be reproduced. A single aggregate number does not preserve this information.

For the application that motivated the work, the practical conclusions are specific. YOLO11n and YOLO11s meet a 30 FPS camera budget on the smartphone's Neural Engine with three to four times the required margin at their p90 latency. Models above YOLO11m do not improve accuracy, so additional capacity does not address the measured limitations of this system. The default HSV jitter should be retained because removing it costs an order of magnitude more than widening it returns. Fully-ripe counting error should be reported alongside mAP because the two metrics do not follow the same pattern. The confidence threshold should be selected on frames that are not used to report the result.

Future work should test the photometric results on a collection recaptured under measured illumination. The current sweep is a simulation applied to existing frames, so the additional collection would show whether the ordering holds with real sensor behaviour. Instance masks would improve the box-overlap estimate of natural occlusion, although they would still not establish depth order. Seed repeats across the whole ladder, beyond the colour recipes, would allow the differences among YOLO11m, l, and x to be ranked instead of reported as indistinguishable. The deployment study also lacks a comparison between a training-time intervention and exposure control at capture time. Such a comparison could determine whether capture control recovers the over-exposure loss at a lower cost than changing the model.

References

[1] G. Kootstra, X. Wang, P. M. Blok, J. Hemming, and E. van Henten, "Selective harvesting robotics: Current research, trends, and future directions," *Current Robotics Reports*, vol. 2, no. 1, pp. 95–104, 2021.

[2] H. Yaguchi, K. Nagahama, T. Hasegawa, and M. Inaba, "Development of an autonomous tomato harvesting robot with rotational plucking gripper," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 652–657.

[3] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629.

[4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[5] Apple Inc., "iOS 11 brings powerful new features to iPhone and iPad this fall," Apple Newsroom, Jun. 2017, accessed: 2026-10-26. [Online]. Available: https://www.apple.com/newsroom/2017/06/ios-11-brings-new-features-to-iphone-and-ipad-this-fall/.

[6] ——, "The future is here: iPhone X," Apple Newsroom, Sep. 2017, accessed: 2026-10-26. [Online]. Available: https://www.apple.com/newsroom/2017/09/the-future-is-here-iphone-x/.

[7] Laboro.AI, "Laboro tomato: Instance segmentation dataset," GitHub repository, 2020, accessed: 2026-09-08. [Online]. Available: https://github.com/laboroai/LaboroTomato

[8] V. Tsironis, S. Bourou, and C. Stentoumis, "tomatOD: Evaluation of object detection algorithms on a new real-world tomato dataset," in *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLIII-B3-2020, 2020, pp. 1077–1084.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[10] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," 2024.

[11] G. Liu, J. C. Nouaze, P. L. Touko Mbouembe, and J. H. Kim, "YOLO-Tomato: A robust algorithm for tomato detection based on YOLOv3," *Sensors*, vol. 20, no. 7, p. 2145, 2020.

[12] F. Su, Y. Zhao, G. Wang, P. Liu, Y. Yan, and L. Zu, "Tomato maturity classification based on SE-YOLOv3-MobileNetV1 network under nature greenhouse environment," *Agronomy*, vol. 12, no. 7, p. 1638, 2022.

[13] R. Li, Z. Ji, S. Hu, X. Huang, J. Yang, and W. Li, "Tomato maturity recognition model based on improved YOLOv5 in greenhouse," *Agronomy*, vol. 13, no. 2, p. 603, 2023.

[14] P. Li, J. Zheng, P. Li, H. Long, M. Li, and L. Gao, "Tomato maturity detection and counting model based on MHSA-YOLOv8," *Sensors*, vol. 23, no. 15, p. 6701, 2023.

[15] J. C. Camacho and M. E. Morocho-Cayamcela, "Mask R-CNN and YOLOv8 comparison to perform tomato maturity recognition task," in *Communication and Intelligent Systems*, ser. Communications in Computer and Information Science, vol. 1885, 2023, pp. 382–396.

[16] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. PMLR, vol. 70, 2017, pp. 1321–1330.

[17] T. Zeng, S. Li, Q. Song, F. Zhong, and X. Wei, "Lightweight tomato real-time detection method based on improved YOLO and mobile deployment," *Computers and Electronics in Agriculture*, vol. 205, p. 107625, 2023.

[18] L. Jia, X. Yuan, Z. Chen, T. Wang, L. Gao, G. Gu, X. Wang, and Y. Wang, "TSE-YOLO: A model for tomato ripeness segmentation," *Agriculture*, vol. 16, no. 1, p. 8, 2025.

[19] X. Liu, W. Teng, H. Yu, Z. Yao, C. Wang, Y. Peng, X. Han, and J. Liu, "GAE-YOLO: A lightweight multimodal detection framework for tomato smart agriculture with edge computing," *Frontiers in Plant Science*, vol. 16, p. 1712432, 2025.

[20] A. I. Kafi, A. P. Sanfilippo, R. Jovanovic, and S. A.-H. Shannak, "Drone-based tomato fruit detection through hardware-accelerated YOLO deployment," in *Proceedings of the Tenth International Congress on Information and Communication Technology (ICICT 2025)*, ser. Lecture Notes in Networks and Systems, vol. 1415. Springer, 2025, pp. 507–519.

[21] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems*, vol. 3, 2021.

[22] United States Department of Agriculture, Agricultural Marketing Service, "United states standards for grades of fresh tomatoes," 1991, reprinted 1997; colour classification also in 7 CFR § 51.1860. [Online]. Available: https://www.ams.usda.gov/sites/default/files/media/Tomato_Standard[1].pdf.

[23] M. Afonso, H. Fonteijn, F. S. Fiorentin, D. Lensink, M. Mooij, N. Faber, G. Polder, and R. Wehrens, "Tomato fruit detection and counting in greenhouses using deep learning," *Frontiers in Plant Science*, vol. 11, p. 571299, 2020.

[24] Z.-w. Wu, M.-h. Liu, C.-x. Sun, and X.-f. Wang, "A dataset of tomato fruits images for object detection in the complex lighting environment of plant factories," *Data in Brief*, vol. 48, p. 109291, 2023.

[25] Make ML, "Tomato detection dataset," Kaggle, 2020, accessed: 2026-10-26. [Online]. Available: https://www.kaggle.com/datasets/andrewmvd/tomato-detection.

[26] Y. Mu, T.-S. Chen, S. Ninomiya, and W. Guo, "Intact detection of highly occluded immature tomatoes on plants using deep learning techniques," *Sensors*, vol. 20, no. 10, p. 2984, 2020.

[27] A. F. López Camelo and P. A. Gómez, "Comparison of color indexes for tomato ripening," *Horticultura Brasileira*, vol. 22, no. 3, pp. 534–537, 2004.

[28] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024, software, version 11.

[29] A. Neubeck and L. Van Gool, "Efficient non-maximum suppression," in *Proceedings of the 18th International Conference on Pattern Recognition (ICPR)*, 2006, pp. 850–855.

[30] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision*, 2014, pp. 740–755.

[31] N. Häni, P. Roy, and V. Isler, "A comparative study of fruit detection and counting methods for yield mapping in apple orchards," *Journal of Field Robotics*, vol. 37, no. 2, pp. 263–282, 2020.

[32] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2011, pp. 1521–1528.

[33] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *International Conference on Learning Representations*, 2019.

[34] M. Afifi and M. S. Brown, "What else can fool deep learning? Addressing color constancy errors on deep neural network performance," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 243–252.

[35] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 60, 2019.

[36] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv:2004.10934, 2020.

[37] Ultralytics, "Configuration: Augmentation settings and hyperparameters," Ultralytics YOLO Docs, 2026, accessed: 2026-10-26. [Online]. Available: https://docs.ultralytics.com/usage/cfg/.

[38] R. Kirk, G. Cielniak, and M. Mangan, "L*a*b*fruits: A rapid and robust outdoor fruit detection system combining bio-inspired features with one-stage deep learning networks," *Sensors*, vol. 20, no. 1, p. 275, 2020.

[39] S. Haggag, M. Veres, C. Tarry, and M. Moussa, "Object detection in tomato greenhouses: A study on model generalization," *Agriculture*, vol. 14, no. 2, p. 173, 2024.

[40] A. Wang, Y. Sun, A. Kortylewski, and A. L. Yuille, "Robust object detection under occlusion with context-aware compositionalnets," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 12 645–12 654.

[41] S. Gilroy, D. Mullins, A. Parsi, E. Jones, and M. Glavin, "Replacing the human driver: An objective benchmark for occluded pedestrian detection," *Biomimetic Intelligence and Robotics*, vol. 3, no. 3, p. 100115, 2023.

[42] K. Kassaw, F. Luzi, L. M. Collins, and J. M. Malof, "Are deep learning models robust to partial object occlusion in visual recognition tasks?" *Pattern Recognition*, vol. 171, p. 112215, 2026.

[43] T. DeVries and G. W. Taylor, "Improved regularization of convolutional neural networks with Cutout," arXiv:1708.04552, 2017.

[44] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 07, 2020, pp. 13 001–13 008.

[45] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 6023–6032.

[46] T. Liao, J. Villacrés, and S. G. Vougioukas, "Improved amodal fruit segmentation via data augmentation and Mask R-CNN head modification," *IFAC-PapersOnLine*, no. 23, pp. 413–418, 2025.

[47] A. Shaheen and O. F. El-Gayar, "Prospects of deep learning and edge intelligence in agriculture: A review," in *Perspectives and Considerations on the Evolution of Smart Systems*. IGI Global, 2023.

[48] A. Ignatov, R. Timofte, A. Kulik, S. Yang, K. Wang, F. Baum, M. Wu, L. Xu, and L. Van Gool, "AI benchmark: All about deep learning on smartphones in 2019," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, 2019, pp. 3617–3635.

[49] V. Mazzia, A. Khaliq, F. Salvetti, and M. Chiaberge, "Real-time apple detection system using embedded systems with hardware accelerators: An edge AI application," *IEEE Access*, vol. 8, pp. 9102–9114, 2020.

[50] J. C. F. da Silva, M. C. Silva, E. J. S. Luz, S. Delabrida, and R. A. R. Oliveira, "Using mobile edge AI to detect and map diseases in citrus orchards," *Sensors*, vol. 23, no. 4, p. 2165, 2023.

[51] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Scaled-YOLOv4: Scaling cross stage partial network," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 13 029–13 038.

[52] CIE, "Colorimetry, 4th edition," International Commission on Illumination, Vienna, Tech. Rep. CIE 015:2018, 2018.

[53] M. Afifi, K. G. Derpanis, B. Ommer, and M. S. Brown, "Learning multi-scale photo exposure correction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 9153–9163.

[54] T. Brooks, B. Mildenhall, T. Xue, J. Chen, D. Sharlet, and J. T. Barron, "Unprocessing images for learned raw denoising," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 11 028–11 037.

[55] Apple Inc., "Measuring your app's power use with power profiler," Apple Developer Documentation, 2026, accessed: 2026-09-08. [Online]. Available: https://developer.apple.com/documentation/xcode/measuring-your-app-s-power-use-with-power-profiler

APPENDIX A
SOURCE CODE, DATA, AND ARTEFACTS

All source code, configuration, and the scripts that generate every table and figure in this paper are available at:

https://github.com/hellothereabc/final_project

The repository contains the experiment code and the iOS application used for the on-device measurements. It also contains a README with build and run instructions, this paper in PDF form, and a short screencast of the application running on a physical device. Section VI describes the provenance of the reported numbers.